

# UNIVERSIDADE FEDERAL DE MINAS GERAIS

Autores: Felipe Aoun Resende

24 de maio de 2026

# Trabalho EDB

$$x(t) \longrightarrow \boxed{h(t)} \longrightarrow y(t)$$

$x$  = Entrada

$h$  = sistema

$y$  = resposta

$$y(t) = h(t) * x(t)$$

$$Y(j\omega) = H(j\omega) \cdot X(j\omega)$$

$$Y(j\omega) \xrightarrow{\mathcal{F}^{-1}} y(t)$$

Ao invés de fazer a convolução é possível fazer a multiplicação do sistema e da entrada com transformadas de Fourier e depois fazer a T. inversa F.

A ideia é pegar o mesmo sistema (Ex: Amp Op) e fazer a mesma função de transferência, uma por T.F. e outra por convolução. Ver se há vantagem em tempo para o computador.

A função de transferência importa, pois em uma mais simples a diferença nem importa.

Em T.F. temos o gráfico do espectro de frequência ver quais funções compõe a função de transferência. Mostrar que não podemos fazer essa análise no tempo. Ver se a frequência do sinal de entrada é amplificada ou reduzida.

## 1. Os filtros

Nesta etapa são considerados dois filtros para comparar a convolução direta com o processamento no domínio da frequência: um filtro RC de 1ª ordem e um filtro Butterworth aproximado de 4ª ordem.

### 1.1 Filtro RC

O filtro RC é o sistema mais simples analisado. A função de transferência exibida é

$$H(s) = \frac{10000}{s + 10000}$$

pois:

Numerador:

[10000.0]

Denominador:

[1.0, 10000.0]

O polo está em

$$s = -10000$$

e a constante de tempo é

$$\tau = \frac{1}{10000} = 100 \mu s$$

exatamente o esperado para:

R = 1e3  
C = 100e-9

porque

$$RC = 1000 \cdot 100 \times 10^{-9} = 10^{-4} \text{ s}$$

No domínio da frequência, usando  $s = j\omega$ , a função de transferência do filtro RC pode ser escrita como:

$$H(j\omega) = \frac{1}{1 + j\omega RC}$$

No código, essa expressão aparece como:

```
h_fourier = (  
    "H(jw) = 1 / (1 + jwRC) "  
)
```

Esse filtro serve como referência porque possui apenas uma constante de tempo e uma resposta impulsiva mais simples.

## 1.2 Filtro Butterworth de 4ª Ordem

O filtro Butterworth utilizado no projeto é usado como sistema mais complexo, aproximado por uma função de transferência de 4ª ordem. Ele permite verificar se um sistema com maior ordem altera significativamente o desempenho da convolução direta e da FFT.

No código, esse sistema é criado pela função:

```
def sistema_complexo():  
    fc = 1000  
    wc = 2*np.pi*fc  
  
    num, den = signal.butter(  
        N=4,  
        Wn=wc,  
        analog=True  
    )  
  
    return signal.TransferFunction(num, den)
```

Inicialmente é definida a frequência de corte do filtro:

$$f_c = 1000 \text{ Hz}$$

Como a função `signal.butter()` utiliza frequência angular para filtros analógicos, é necessário converter a frequência de corte para rad/s:

$$\omega_c = 2\pi f_c$$

Substituindo  $f_c = 1000 \text{ Hz}$ :

$$\omega_c = 2\pi(1000) \approx 6283.19 \text{ rad s}^{-1}$$

Em seguida, a função `signal.butter()` projeta automaticamente um filtro Butterworth passa-baixas de quarta ordem:

```
num, den = signal.butter(
    N=4,
    Wn=wc,
    analog=True
)
```

onde:

- `N=4` define um filtro de quarta ordem;
- `Wn=wc` define a frequência de corte;
- `analog=True` indica que o filtro será representado no domínio contínuo, produzindo uma função de transferência  $H(s)$ .

O SciPy retorna dois vetores:

- **num**: coeficientes do numerador da função de transferência;
- **den**: coeficientes do denominador da função de transferência.

Para uma frequência de corte de 1000 Hz, um resultado típico é:

Numerador:  
[1.5585e15]

Denominador:  
[  
1.0,  
1.6419e4,  
1.3479e8,  
6.4819e11,  
1.5585e15  
]

Isso corresponde aproximadamente à função de transferência:

$$H(s) = \frac{1.5585 \times 10^{15}}{s^4 + 1.6419 \times 10^4 s^3 + 1.3479 \times 10^8 s^2 + 6.4819 \times 10^{11} s + 1.5585 \times 10^{15}}$$

Por fim, os coeficientes são convertidos em um objeto do tipo **TransferFunction**:

```
return signal.TransferFunction(num, den)
```

Esse objeto é posteriormente utilizado para calcular a resposta impulsiva do sistema  $h(t)$ , que será empregada tanto na convolução direta quanto na convolução utilizando FFT. Dessa forma, é possível comparar o desempenho computacional dos dois métodos e verificar que ambos produzem praticamente a mesma resposta para o sistema analisado.

## Representação em Fourier

Para análise no domínio da frequência, substitui-se:

$$s = j\omega$$

obtendo:

$$H(j\omega) = \frac{1.5585 \times 10^{15}}{D(j\omega)}$$
$$D(j\omega) = (j\omega)^4 + 1.6419 \times 10^4 (j\omega)^3 + 1.3479 \times 10^8 (j\omega)^2 + 6.4819 \times 10^{11} (j\omega) + 1.5585 \times 10^{15}$$

De forma geral, essa mesma expressão pode ser escrita como:

$$H(j\omega) = \frac{b_0}{(j\omega)^4 + a_1(j\omega)^3 + a_2(j\omega)^2 + a_3(j\omega) + a_4}$$

No código, essa representação aparece como:

```
h_fourier = (
    "H(jw) = b0 / ((jw)^4 + a1(jw)^3 + a2(jw)^2 + a3(jw) + a4)"
)
```

Essa expressão descreve como cada componente espectral do sinal de entrada é amplificada ou atenuada pelo filtro Butterworth. Como se trata de um filtro passa-baixas, frequências abaixo de 1000 Hz sofrem pouca atenuação, enquanto frequências acima da frequência de corte são progressivamente reduzidas.

A principal diferença em relação ao filtro RC é que o Butterworth possui mais coeficientes e uma dinâmica de ordem maior. Por isso, ele é útil para observar se a complexidade algébrica da função de transferência influencia o tempo de processamento ou se o fator dominante continua sendo o número de amostras do sinal.

## 2) Análise de Desempenho: Convolução Direta vs. FFT

No seu código, o speedup é calculado como:

$$\text{Speedup} = \frac{t_{conv}}{t_{FFT}}$$

Portanto:

- **Speedup** > 1 → FFT foi mais rápida.
- **Speedup** = 1 → mesmo tempo.
- **Speedup** < 1 → convolução direta foi mais rápida.

Se apareceu:

Speedup:

571.62x

significa que

$$t_{conv} = 571,62 \times t_{FFT}$$

ou seja, a convolução direta demorou 571,62 vezes mais tempo que a convolução por FFT.

Por exemplo, algo como:

Tempo Conv. : 0.571620 s

Tempo FFT : 0.001000 s

produziria:

$$\frac{0,571620}{0,001000} = 571,62$$

Logo, para esse teste, a Transformada de Fourier (FFT) foi aproximadamente **572 vezes mais rápida**.

Isso é exatamente o comportamento esperado quando o número de amostras é grande. A convolução direta tem complexidade aproximadamente:

$$O(N^2)$$

enquanto a convolução por FFT tem complexidade:

$$O(N \log N)$$

Por isso, conforme o sinal cresce, a FFT tende a superar a convolução direta por uma margem cada vez maior. No seu benchmark, os casos com poucas amostras (por exemplo, 1000 amostras) provavelmente apresentam speedup menor que 1, mas nos maiores testes (50.000 amostras ou mais) é normal encontrar speedups de centenas de vezes.

### 3) Amp Op - Amplificador Operacional

É um circuito integrado de altíssimo ganho que amplifica a diferença de tensão entre duas entradas. É um bloco muito versátil de eletrônica analógica, utilizado em amplificadores, filtros, osciladores etc.

- **Entrada inversora (-):** Um sinal aplicado gera uma saída com fase invertida.
- **Entrada não-inversora (+):** Mantém a fase original na saída.

$$|X(j\omega)|e^{j\angle X(j\omega)} \quad \text{ou} \quad Z = a + bj$$

$$\sqrt{a^2 + b^2} = \text{Módulo}$$

$$\arctan(b/a) = \text{Fase}$$

- **Módulo** informa se o sinal foi amplificado ou atenuado.
- **A fase** indica se o sinal foi atrasado ou adiantado.
- **Saída:** Fornece a diferença entre os sinais das entradas multiplicados pelo ganho do amplificador.

$$\begin{array}{c} V_+ \longrightarrow \backslash \\ V_- \longrightarrow / \longrightarrow V_{out} \end{array}$$

### 4) Função de Transferência

$$H(j\omega) = \frac{Y(j\omega)}{X(j\omega)}$$

- **Polos:** Definem a estabilidade e velocidade do sistema.
- **Zeros:** Anulam certas frequências ou aceleram a resposta.

### 5) Análise dos resultados dos gráficos e do tempo de execução entre T.F. e convolução direta

Nesta etapa, a análise compara a convolução direta com o processamento por Transformada de Fourier (FFT), observando tempo de execução, precisão numérica e influência do tamanho do sinal.

#### 5.1 Sistemas analisados

Foram considerados dois tipos de sistema:

- **Sistema simples:** filtro passa-baixas RC de 1ª ordem;
- **Sistema complexo:** filtro ativo Butterworth aproximado de 4ª ordem.

Um exemplo de função de transferência utilizada no teste é dado por:

Numerador:

[1.0]

Denominador:

[0.0001, 1.0]

A comparação entre esses dois sistemas permite observar se a ordem do filtro altera significativamente o tempo de processamento ou se o fator dominante é o número de amostras.

## 5.2 Configuração do teste

A configuração básica usada nos testes foi:

| Parâmetro                | Valor      |
|--------------------------|------------|
| Frequência de amostragem | 100 000 Hz |
| Duração                  | 0.10 s     |
| Número de amostras       | 10000      |

O sinal de entrada foi composto por três senoides somadas a ruído branco:

$$x(t) = \sin(2\pi 100t) + 0.5 \sin(2\pi 500t) + 0.25 \sin(2\pi 2000t) + \text{ruído branco}$$

## 5.3 Resultados de desempenho

Para o caso com 10000 amostras, os resultados obtidos foram:

| Métrica                    | Valor                  |
|----------------------------|------------------------|
| Tempo da convolução direta | 0.004 251 s            |
| Tempo da FFT               | 0.000 721 s            |
| Speedup                    | 5.89×                  |
| Erro RMS                   | $2.17 \times 10^{-14}$ |

O erro RMS mostra que os métodos produziram praticamente a mesma saída. Assim, a principal diferença entre eles não está na precisão do resultado, mas no custo computacional.

## 5.4 Comportamento com o tamanho do sinal

O comportamento observado depende diretamente do número de amostras:

- Para sinais pequenos, a convolução direta pode ser mais rápida;
- Para sinais médios, a FFT começa a apresentar vantagem;
- Para sinais grandes, a FFT domina completamente.

Para  $N = 1000$ , por exemplo, a FFT pode ser mais lenta, com speedups como  $0.25\times$  ou  $0.68\times$ . Isso ocorre porque existe um custo inicial associado às operações de `fft()`, `ifft()`, preenchimento com zeros e multiplicação espectral. Já a convolução direta é simples e curta para poucos pontos.

Para sinais longos, o comportamento se inverte. Em um caso com aproximadamente 100000 amostras, espera-se algo próximo de:

| Método | Tempo  |
|--------|--------|
| Manual | 40 s   |
| NumPy  | 0.7 s  |
| FFT    | 0.02 s |

e erros da ordem de  $10^{-15}$ , indicando que os métodos permanecem numericamente equivalentes, mas com custos computacionais muito diferentes.

## 5.5 Comparação entre métodos

A análise fica mais completa quando compara três abordagens:

- Implementação matemática pura, usando convolução manual;
- Biblioteca otimizada, usando `np.convolve`;
- Método espectral baseado em Fourier, usando FFT.

As complexidades teóricas esperadas são:

| Método             | Complexidade  |
|--------------------|---------------|
| DFT manual         | $O(N^2)$      |
| FFT NumPy          | $O(N \log N)$ |
| Convolução direta  | $O(N^2)$      |
| Convolução via FFT | $O(N \log N)$ |

Essa comparação mostra claramente o ganho algorítmico da FFT em relação aos métodos quadráticos, principalmente quando o número de amostras aumenta.

## 5.6 Interpretação dos resultados

Os resultados dos dois sistemas analisados, filtro RC e filtro Butterworth de 4ª ordem, foram praticamente idênticos em termos de tempo de execução. Isso indica que o custo computacional da convolução depende muito mais do tamanho dos vetores do que da complexidade algébrica da função de transferência.

Assim, uma conclusão importante para o relatório é que a troca de um filtro RC de primeira ordem por um Butterworth de quarta ordem não alterou significativamente o tempo de processamento da convolução nem da FFT. O fator predominante foi a quantidade de amostras processadas.

O programa, portanto, cumpre os principais objetivos da análise:

- Mostra explicitamente a função de transferência utilizada;
- Mostra a entrada utilizada;
- Mostra amostras da saída;
- Calcula o erro RMS;
- Compara NumPy  $\times$  FFT;
- Plota entrada, resposta impulsiva, saída e espectro;

- Executa os testes para vários tamanhos de sinal;
- Mantém o foco na comparação entre convolução direta e FFT em sistemas de Amp Op.

A convolução manual não deve ficar ativa por padrão, pois com 50000 amostras ela pode ficar extremamente lenta e atrapalhar os testes de desempenho.

## 5.7 Conclusão da análise

A convolução direta apresenta custo quadrático  $O(N^2)$ , tornando-se rapidamente inviável para sinais longos. A implementação baseada na Transformada Rápida de Fourier reduz a complexidade para  $O(N \log N)$ , proporcionando ganhos de desempenho que crescem com o tamanho do sinal.

A vantagem da FFT torna-se ainda mais evidente em filtros ativos de ordem elevada, cuja resposta impulsiva possui maior duração temporal. Para simulações extensas de amplificadores operacionais, o processamento no domínio da frequência é geralmente a abordagem mais eficiente.